
ISEC3004 – Assignment 1

Vulnerability Analysis, Detection, Tracing and Mitigation on Log Injection and Cross Site Request Forgery

Date Submitted : 09.10.2026
Group : 06
Workshop : Monday 8.00am

Group Members	
N P Sasindu Pabasara Samaranayake	22716664
A Siluni Vihansa Deshapriya	22716541
W.M .Santhosh R.J. Perera	22615004

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Document Conventions.....	4
1.3 Scope.....	5
2. Team Organisation and Project Plan.....	5
2.1 Roles and Responsibilities.....	5
2.2 Timeline and Milestones.....	6
2.3 Tools and Workflow.....	6
2.4 Effort Estimation.....	7
2.5 Project Monitoring and Quality Assurance.....	7
3. Vulnerability 1: CSRF.....	8
3.1 Explanation.....	8
3.2 Vulnerable Code.....	8
3.3 Exploitation.....	9
3.4 Detection and Tracing.....	11
3.4.1 Approach.....	11
3.4.2 Runtime detection - request logging.....	11
3.4.3 Baseline legitimate transfer.....	12
3.4.4 Attack - forged cross site transfer.....	12
3.4.5 Impact confirmed via account balances.....	12
3.4.6 Browser based detection(Chrome DevTools).....	13
3.4.7 Proxy Based interception capture (Burp).....	14
3.4.8 Summary.....	15
3.5 Mitigation and Verification.....	16
3.5.1. Root cause recap.....	16
3.5.2. Before and After verification.....	16
3.5.3 Confirming the fix does not break legitimate use.....	17
3.5.4. Why this mitigation is useful.....	17
4. Vulnerability 2: Log Injection.....	18
4.1 Explanation.....	18
4.2 Vulnerable Code.....	19
4.3 Exploitation.....	19

4.4 Detection and Tracing.....	20
4.4.1 Approach.....	20
4.4.2 Baseline - legitimate log entries.....	21
4.4.3 Attack - forged log entry via injection.....	21
4.4.4 Detection - identifying the forged entry.....	22
4.4.5 Real world considerations.....	22
4.4.6 Summary.....	23
4.5 Mitigation and Verification.....	23
4.5.1 Mitigation technique.....	23
4.5.2 Before and After Verification.....	24
4.5.3 Confirming the fix does not break legitimate use.....	25
4.5.4 Why this mitigation is effective.....	25
5. Mitigation Strategies and Security Recommendations.....	27
6. Evidence of Project Management.....	28
6.1 Project artifacts and links.....	28
6.2 Task allocation and Jira Board.....	28
6.3 Schedule and milestones.....	29
6.4 Version Control.....	30
6.5 Effort Estimation.....	31
6.6 Meeting Records.....	32
6.7 Quality Assurance.....	32
6.8 Communication.....	33
7. Conclusion.....	35
8. References.....	35
9. Appendices.....	37
Appendix A.....	37
Appendix B.....	38
Appendix C.....	38
Appendix D.....	40
Appendix E.....	40
Appendix F.....	40
Appendix G.....	40

1. Introduction

1.1 Purpose

This report documents our group work done for the ISEC3004 Assignment 1 security group project. As per the specification required, we selected two vulnerabilities from the approved list and built the vulnerable code demonstrating each flaw, exploited it, detected and traced the resulting malicious activity, implemented and verified mitigations and managed the whole exercise through proper project management practices.

The Two vulnerabilities that were selected were the Log Injection and the Cross-Site Request Forger(CSRF). These two vulnerabilities were chosen because, together they cover complementary parts of a web application's attack surface while remaining achievable for a three person team within the assignment timeframe.

Log Injection was selected because it targets the integrity of the evidence trail itself where an attacker who can forge or corrupt log entries can hide their own activity or frame another user, which made it a natural fit for a project that also has a strong detection and tracing marking component. Writing unvalidated user input to log files can allow an attacker to forge log entries or inject malicious content into the logs and this is called log injection. This directly threatens the reliability of the very tools of the system and the application logs.

CSRF was chosen as the second vulnerability because it demonstrates a different class of problem as an attack on trust between the browser and an authenticated session rather than on unsanitised input reaching a data sink. CSRF attacks take advantage of the trust a website has already granted to an authenticated user and their browser. Tricking that user into unknowingly executing an unwanted action. It is also a well documented historically significant vulnerability as CSRF first appeared in the OWASP top 10 in 2007 and remained on the list through 2013 and even though it was later merged into the broader broken Access Control category it remains a live threat in legacy applications, custom frameworks and anywhere developers disable built in framework protections which make it a realistic and current choice rather than a purely historical one.

Together, the two vulnerabilities let the group show breadth, as one is an injection-style flaw affecting backend logging integrity and the other a session/trust based flaw affecting front-end request authenticity. The process is well documented and fully verified in this report.

1.2 Document Conventions

This report uses decimal heading numbers throughout (3.2, 3.2). Each of the two vulnerability sections follows the same five parts, namely the explanation, vulnerability code, exploitation, detection and tracing and mitigation and verification. This structure allows the two case studies to be read and compared consistently. Code is presented in monospaced code blocks with inline comments marking the specific lines responsible for the vulnerability or the fix. Terminal output, log entries and tool captures are presented as screenshots or plain text blocks immediately followed by a

short interpretation explaining what the evidence proves. Team evidence (Jira board, GitHub and meeting notes) are presented in section 5.

1.3 Scope

This report covers the investigation, exploitation, detection and mitigation of two vulnerabilities namely Log Injection and Cross Site Request Forgery (CSRF). For each of these two vulnerabilities, the project encompasses the design and implementation of a deliberately vulnerable program, the development of a reproducible exploitation procedure demonstrating how the flaw can be abused, the detection and tracing of the resulting malicious activity using appropriate security tools and the design, implementation and verification of an effective mitigation for each with before and after testing confirming that the original exploit no longer succeeds once the fix is applied. This report excludes the development of production grade or performance optimised implementations of the mitigation techniques as the code produced is intent to demonstrate the correct application of security principles rather than to serve as deployment ready software and it excludes vulnerabilities that may exist third party frameworks, libraries or infrastructure components that fall outside the group's own demonstration code.

2. Team Organisation and Project Plan

2.1 Roles and Responsibilities

Group 6 has three members and we developed both vulnerabilities collaboratively rather than dividing them one per person. To keep accountability clear while preserving shared involvement, each work area has a designated lead who holds final responsibility for that deliverable, supported by the other members who assist, review and validate the work.

Work Area	Difficulty (0-5)	Sasindu	Siluni	Santhosh
Jira / Google Drive / Meeting Notes management	1	Lead	Support	Support
GitHub repo & version control	1	Support	Lead	Support
Vuln 1 (CSRF) - code	4	Lead	Support	Support
Vuln 1 - exploit	3	Lead	Support	Support
Vuln 1 - detection & tracing	3	Support	Support	Lead
Vuln 1 - mitigation	4	Support	Lead	Support
Vuln 1 - report write-up	2	Lead	Support	Support
Vuln 2 (Log Injection) - code	4	Support	Lead	Support

Vuln 2 - exploit	3	Support	Lead	Support
Vuln 2 - detection & tracing	3	Lead	Support	Support
Vuln 2 - mitigation	4	Support	Support	Lead
Vuln 2 - report write-up	2	Support	Support	Lead
QA / testing (code review, exploit validation, mitigation re-test)	2	Lead	Lead	Lead
Report (intro, conclusion, discussions, editing, Project Management evidence)	3	Support	Support	Lead
Presentation/slides	3	Lead	Lead	Lead

2.2 Timeline and Milestones

Work is scheduled around a mid-project checkpoint so that any problems with the harder vulnerability (CSRF) surface early enough to recover. The milestones below give the broad schedule; detailed task due dates, owners and progress are maintained on the group's Jira board.

Date	Milestone
15 Sep	Group formed; vulnerabilities chosen; tools and roles set
19 Sep	GitHub repo and Jira board fully set up; starter tasks assigned
2 Oct	Both vulnerable programs built and exploits working
4 Oct	Detection/tracing evidence captured; mitigations implemented and verified
6 Oct	Full report draft complete; internal review begins
9 Oct	Report and code submitted
Week 12	Live presentation in lab session

2.3 Tools and Workflow

The team uses a dedicated toolset to coordinate work and maintain a version-controlled project record. Jira is used for task allocation, effort estimation and progress tracking via a Kanban board. GitHub hosts all code, with each team member committing their own work. Google Drive stores collaborative documents, including meeting notes and the report. WhatsApp is used for day-to-day communication and scheduling.

Tool	Purpose
Jira	Task allocation, effort estimation and progress tracking via a Kanban board
GitHub	Version control and code hosting; each member commits their own work, providing a record of individual contribution
Google Drive	Storage for collaborative documents, including meeting notes and the report
WhatsApp	Day-to-day communication and scheduling

2.4 Effort Estimation

Major tasks were estimated in hours on the Jira board before work began, so we could check the total workload and balance it across team members. Estimates were made collaboratively during planning meetings, with each task sized roughly into hour buckets (1h, 2h, 3h, 4h, 5h) rather than exact hours. The technical pipeline for each vulnerability (research, coding, exploitation, tracing, mitigation and verification) was estimated at approximately 20 hours per vulnerability.

The full breakdown by member is shown below.

Member	Estimated Hours
Sasindu	18 hours
Siluni	19 hours
Santhosh	18 hours
Total	~55 hours

Detailed per-task estimates are maintained on the Jira board (Section 5).

2.5 Project Monitoring and Quality Assurance

Progress was monitored continuously through the Jira board, where task status (completed, in progress, in review, to do) was visible to all members. Quality assurance was built into the workflow rather than left to the end. Code was peer-reviewed via GitHub before being merged and each exploit was validated by a second member to confirm it reproduced reliably. Because the two builders each validate the other's vulnerability, quality checks are carried out by someone other than the original author.

3. Vulnerability 1: CSRF

3.1 Explanation

Cross-site request forgery (CSRF) is a web security vulnerability in which attackers manipulate an authenticated user's session by making their browser submit state-changing requests to a secure application without the user's knowledge.

CSRF works by abusing the trust a site places in a logged-in user's browser. The victim must first be authenticated to the target site, so the browser is holding a valid session cookie for that website. While the user is logged in to the target site, the attacker lures the victim into a malicious page via a crafted link, rogue email, or a malicious form. That page silently issues a request to the target site. Because the browser automatically attaches the user's session cookie to any request sent to that site, the forged request arrives with valid credentials and the application, unable to tell if it was not intentional, trusts it and performs the action.

A natural question is why the browser's same-origin policy (SOP) does not stop the malicious site from talking to the target site anyway. The reason is that the SOP only restricts a malicious site from reading the response of a cross-site request; it does not stop any requests from being sent. It is normal web behaviour. Because CSRF is a "write" attack, the attacker does not need a response; the damage is done as soon as the request goes through. Furthermore, the browser attaches cookies based on the destination, not the origin, so any requests sent to the target site automatically include the victim's valid session cookie regardless of its origin. The application therefore receives a fully authenticated request and cannot distinguish it from a legitimate one.

Common consequences of CSRF are unauthorised fund transfers or purchases, changing account credentials, email addresses, or passwords, or, in the worst case, complete account takeover or administrative state changes.

CSRF is significant and widely exploited because it results from missing protections rather than bad code. Because it doesn't cause any visible malfunction, the victim may not realise anything happened until later. While modern frameworks now include default protection, it persists in legacy systems, custom frameworks and whenever developers disable those defences.

3.2 Vulnerable Code

The vulnerability lies in the /transfer route. It authorises the transfer using only the session cookie. There is no CSRF token or origin check.

The exact vulnerable code snippet is attached below. To view the full document, visit the GitHub repository.

```

app.py

# --- THE VULNERABILITY ---
# This transfer is done using only the session cookie.
# There is NO CSRF token check, so the server cannot tell whether
# this POST came from the real dashboard form or from an attacker.
# Any POST to /transfer with a valid session cookie is
# trusted and executed. this is the CSRF flaw.

@app.route("/transfer", methods=["POST"]) # Transfer code page - Transfer
happens and display status
def transfer():
    if "user" not in session: # if no user in session
        return redirect(url_for("login")) # redirect to /login - GET

    sender = session["user"] # Get the user name from session
    to = request.form["to"] # get the "to account" name from the form
    amount = int(request.form["amount"]) # get the "Ammount" from the form

    if accounts[sender]["balance"] ≥ amount and to in accounts: # check if
user balance ≥ amount transferring AND the "to account" username is in fake
db
        accounts[sender]["balance"] -= amount # decrease the amount from
user
        accounts[to]["balance"] += amount # increase the amount from target
        return f"Transferred ${amount} to {to}. <a
href='/dashboard'>Back</a>" #Confirmation and link to /dashboard

    return "Transfer failed. <a href='/dashboard'>Back</a>" # Transfer fail

```

Because the route trusts any POST carrying a valid session cookie, it cannot distinguish a legitimate transfer (submitted from the dashboard) from a forged one (submitted by an attacker's page). This is the flaw the exploit abuses.

3.3 Exploitation

The exploit is a malicious web page (exploit.html) disguised as a prize giveaway. It contains a hidden form that forges a fund transfer request to the vulnerable banking application. Because the victim is already authenticated, the attack succeeds without the victim knowingly authorising anything.

The exploitation can also be triggered without clicking any buttons, which is a more effective way to exploit a vulnerability because it does not rely on the victim pressing a suspicious button. For demonstration purposes, the exploit only triggers when the victim presses a button on the website.

```
exploit.html

<button onclick="document.getElementById('csrf-form').submit();"
style="padding:15px 30px; font-size:20px; font-weight:bold;
cursor:pointer; background:red; color:white; border:none; border-
radius:5px;">Claim Your Gift</button>

<!-- THE ATTACK -->
<form id="csrf-form" action="http://127.0.0.1:5000/transfer"
method="POST">
  <input type="hidden" name="to" value="attacker">
  <input type="hidden" name="amount" value="100">
</form>
```

How the attack works, step by step:

1. The victim logs into the banking application and their browser stores a valid session cookie.
2. While still logged in, the victim is lured to the attacker's page (exploit.html), disguised as a prize giveaway.
3. After a short delay, a "Claim Your Gift" button appears. Clicking it submits the page's hidden form.
4. The hidden form sends a POST request to <http://127.0.0.1:5000/transfer> with the field values set by the attacker and invisible to the victim.
5. The victim's browser automatically attaches their banking session cookie to this request, because cookies are sent based on the request's destination.
6. The application receives a request with a valid session cookie, so the application processes the request.

A successful CSRF attack on this endpoint allows an attacker to move money out of the victim's bank account without their consent or knowledge. The victim bears the full financial loss. More broadly, the same technique applies to other state-changing actions, such as changing an account's email or password and would allow an attacker to lock the victim out and achieve complete account takeover.

The /admin was only made for easier demonstration purposes. You can see the change in funds before and after the attack.

Admin - All Account Balances	Admin - All Account Balances																
<table><tr><th>Account</th><th>Balance</th></tr><tr><td>victim</td><td>\$1000</td></tr><tr><td>attacker</td><td>\$0</td></tr><tr><td>user</td><td>\$500</td></tr></table> Back to Dashboard	Account	Balance	victim	\$1000	attacker	\$0	user	\$500	<table><tr><th>Account</th><th>Balance</th></tr><tr><td>victim</td><td>\$900</td></tr><tr><td>attacker</td><td>\$100</td></tr><tr><td>user</td><td>\$500</td></tr></table> Back to Dashboard	Account	Balance	victim	\$900	attacker	\$100	user	\$500
Account	Balance																
victim	\$1000																
attacker	\$0																
user	\$500																
Account	Balance																
victim	\$900																
attacker	\$100																
user	\$500																

before

after

3.4 Detection and Tracing

3.4.1 Approach

Detection and tracing for this vulnerability combined two complementary methods, namely runtime logging, which captures evidence of the attack while it is actually happening and static analysis which detects the underlying flaw in the source code before any exploit is run. Browser based inspection(Chrome DevTools) was used as the Third independent source of evidence to corroborate the server side log. Using more than one method meant the finding could be cross checked rather than relying on a single source of evidence.

3.4.2 Runtime detection - request logging

Python's built-in logging module was added to [app.py](#) to record every call to the transfer.log file. Each log entry captures the sender,recipient, amount and critically the origin and the referrer HTTP headers of the request. These two headers are the key evidence where a request submitted from the bank application's own dashboard form will show the app's own origin while a forged request originating from an attacker's page will show the attacker's origin instead, regardless of how convincing the request otherwise looks.

The following is the logging code which was updated in the vulnerable code which was implemented by the team member.

```
app.py

#Detection and Tracing logging
origin = request.headers.get("Origin", "N/A") # get the origin header from the request
referer = request.headers.get("Referer", "N/A") # get the referer header from the request

logging.info(
    f"Transfer Attempt user = {sender}, to={to} amount={amount}"
    f"Origin={origin}, Referer={referer} ip={request.remote_addr}"
)

if accounts[sender]["balance"] ≥ amount and to in accounts: # check if user balance ≥ amount transferring AND the "to
account" username is in fake db
    accounts[sender]["balance"] -= amount # decrease the amount from user
    accounts[to]["balance"] += amount # increase the amount from target
    logging.info(f"Transfer Successful user = {sender}, to={to} amount={amount}")
    return f"Transferred ${amount} to {to}. <a href='/dashboard'>Back</a>" #Confirmation and link to /dashboard

logging.info(f"Transfer Failed user = {sender}, to={to} amount={amount}")
return "Transfer failed. <a href='/dashboard'>Back</a>" # Transfer fail
```

Logging was configured at the top of [app.py](#) to write to a dedicated transfer.log file.

```
app.py

logging.basicConfig(
    filename = "transfer.log",
    level = logging.INFO,
    format = "%(asctime)s - %(message)s"
)
```

3.4.3 Baseline legitimate transfer

To establish what normal behaviour looks like, a legitimate transfer was first performed where, logging in as the victim, \$100 was sent to the user through the app's own dashboard form

```
8 2026-10-01 01:18:20,705 - 127.0.0.1 - - [01/Oct/2026 01:18:20] "GET /login HTTP/1.1" 200 -
9 2026-10-01 01:18:45,700 - 127.0.0.1 - - [01/Oct/2026 01:18:45] "esc[32mPOST /login HTTP/1.1esc[0m" 302 -
10 2026-10-01 01:18:45,711 - 127.0.0.1 - - [01/Oct/2026 01:18:45] "GET /dashboard HTTP/1.1" 200 -
11 2026-10-01 01:18:56,520 - Transfer Attempt user = victim, to=user amount=100Origin=http://127.0.0.1:5000,
12 2026-10-01 01:18:56,520 - Transfer Successful user = victim, to=user amount=100
13 2026-10-01 01:18:56,521 - 127.0.0.1 - - [01/Oct/2026 01:18:56] "POST /transfer HTTP/1.1" 200 -
14 2026-10-01 01:19:15,652 - 127.0.0.1 - - [01/Oct/2026 01:19:15] "GET /dashboard HTTP/1.1" 200 -
15 2026-10-01 01:19:17,587 - 127.0.0.1 - - [01/Oct/2026 01:19:17] "esc[32mGET /logout HTTP/1.1esc[0m" 302 -
```

Normal transaction from the victim account to the user account.

Interpretation over here is that both the origin and referrer fields point to 127.0.0.1:5000 which is the bank application itself. This is the expected signature of a legitimate, same origin request and establishes the baseline against which the attack traffic is compared.

3.4.4 Attack - forged cross site transfer

The attack was then carried out using exploit.html. While still authenticated as victim in the bank app, the victim's browser loaded the attacker's page from a separate origin(127.0.0.1:8000) and clicked "Claim Your Gift", which silently submitted the hidden form to transfer.log.

```
2026-10-01 01:19:17,605 - 127.0.0.1 - - [01/Oct/2026 01:19:17] "GET /login HTTP/1.1" 200 -
2026-10-01 01:21:49,292 - 127.0.0.1 - - [01/Oct/2026 01:21:49] "esc[32mPOST /login HTTP/1.1esc[0m" 302 -
2026-10-01 01:21:49,304 - 127.0.0.1 - - [01/Oct/2026 01:21:49] "GET /dashboard HTTP/1.1" 200 -
2026-10-01 01:24:09,649 - Transfer Attempt user = victim, to=attacker amount=100Origin=http://127.0.0.1:8000, Referer=http://127.0.
2026-10-01 01:24:09,649 - Transfer Successful user = victim, to=attacker amount=100
2026-10-01 01:24:09,650 - 127.0.0.1 - - [01/Oct/2026 01:24:09] "POST /transfer HTTP/1.1" 200 -
2026-10-01 01:24:45,345 - 127.0.0.1 - - [01/Oct/2026 01:24:45] "esc[32mGET /logout HTTP/1.1esc[0m" 302 -
```

Attacked transaction from the victim to the attacker account

Transferred \$100 to attacker. [Back](#)

Screenshot of the bank dashboard after clicking "claim your gift"

Interpretation of this is that this entry is functionally identical to the baseline in every respect the server actually checks(user in session to and amount present) which is exactly the problem. The only field that differs is origins or referrer, now pointing to 127.0.0.1:8000, the attacker's page, not the bank application. Despite this the server still authorised and executed the transfer moving \$100 out of the victim's account with no indication to the victim that anything happened. This confirms the server is authenticating the request using only the session cookie without verifying where the request actually came from which is the defining characteristic of CSRF attack.

3.4.5 Impact confirmed via account balances

The account summary endpoint was used to confirm the real world effect of the forged request

Account	Before	After
victim	\$1000	\$800
attacker	\$0	\$100
user	\$500	\$600

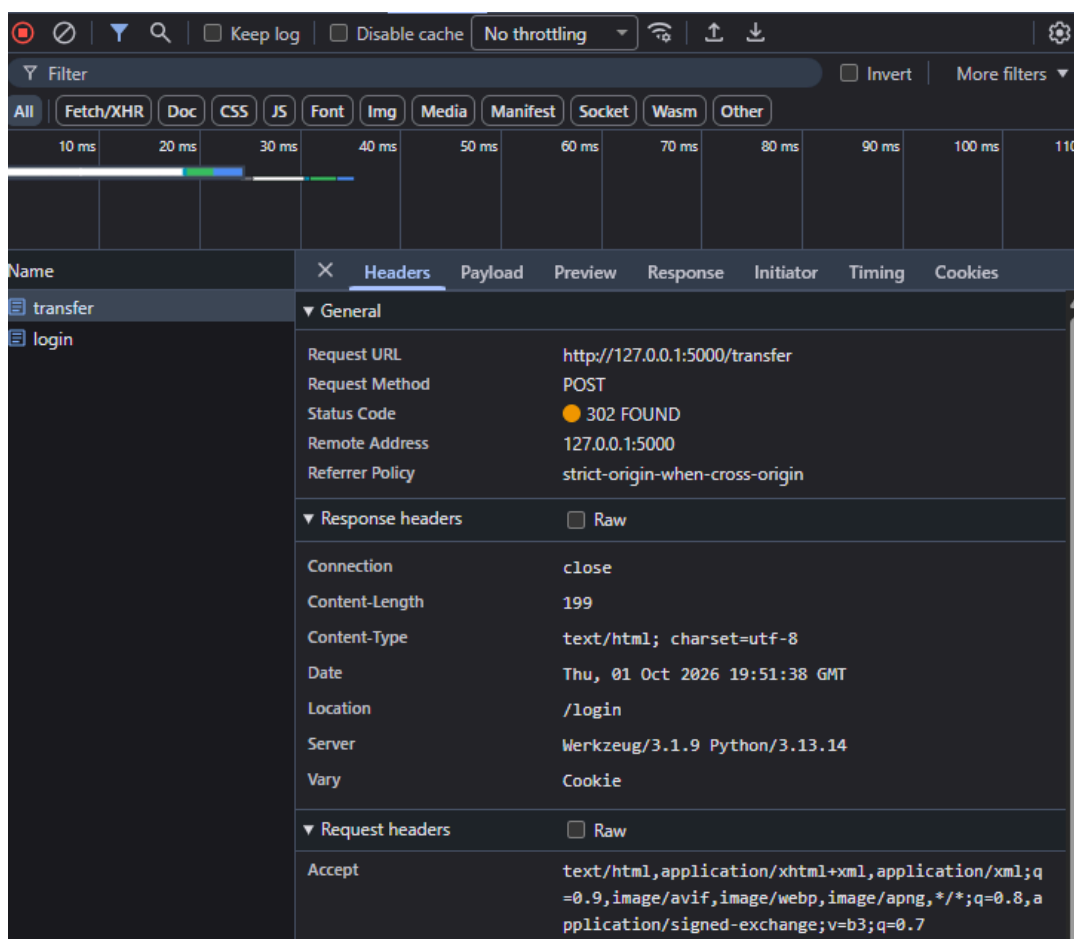
This ties the log evidence directly to real impact where the victim lost funds to the attacker's account without knowingly authorising the specific transfer.

3.4.6 Browser based detection(Chrome DevTools)

As an independent client side source of evidence the attack was also captured using Chrome DevTools.

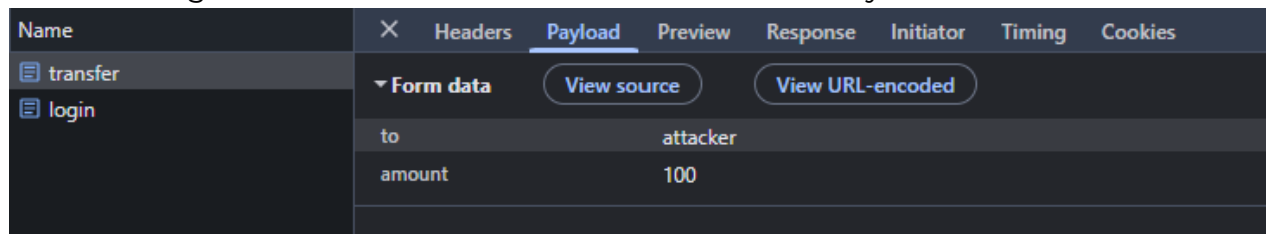
The process was as follows,

1. DevTools opened - Network tab, before clicking "Claim Your gift" on the attacker's page.
2. After clicking the transfer request appears in the request list.
3. Selecting it and viewing the Headers tab shows,
 - a. Request Url :<http://127.0.0.1:5000/transfer>
 - b. Request Method : POST
 - c. The Cookie header showing the browser automatically attached the victim's existing session cookie without any action by the victim beyond the earlier login.
 - d. Origin : http:127.0.0.1:8000

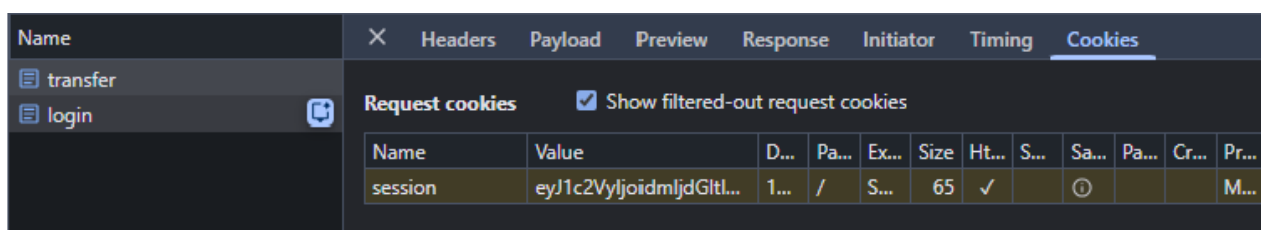


Browser detection of the Transfer using the cookie

Interpretation in this is that this independently confirms what the server log showed where the browser silently attached the victim's authentication cookie to a request initiated entirely by the attacker's page with no involvement or awareness from the victim. The combination of the server log proving the server accepted it) and the DevTools capture (proving the browser sent it automatically) gives two independent, corroborating sources of evidence for the same vulnerability.



DevTools Network section payload screen



DevTools Network section cookie screen

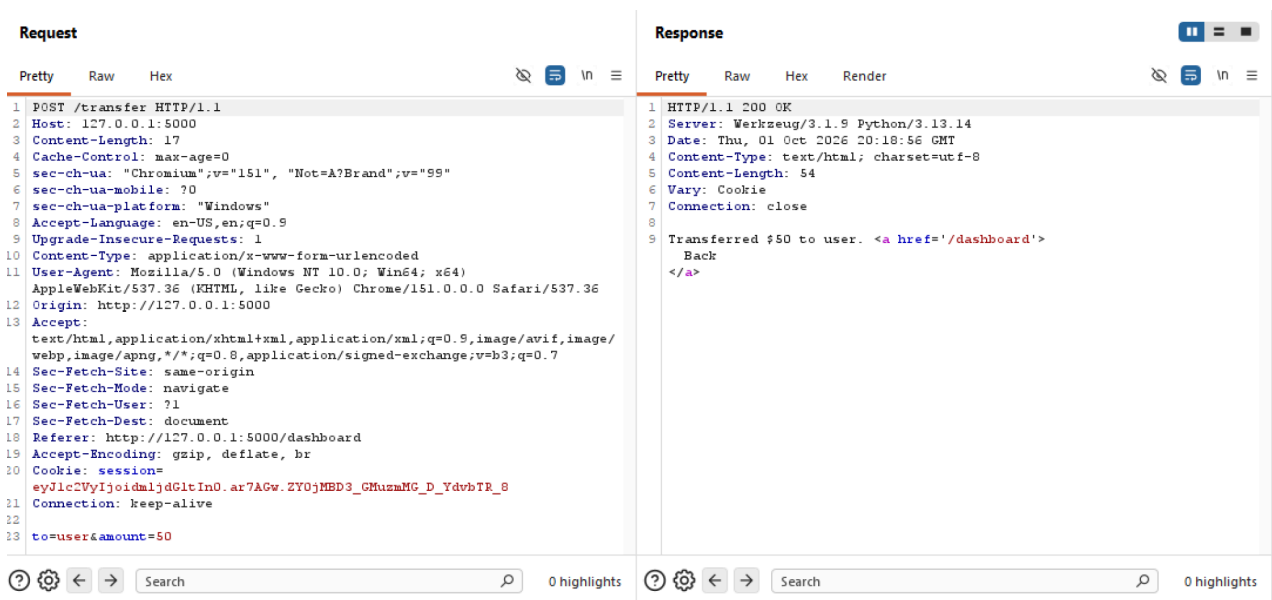
3.4.7 Proxy Based interception capture (Burp)

As the third, independent capture method in addition to server side logging and Chrome DevTools, traffic to transfer was captured using Burp Suite community edition, an intercepting proxy positioned between the browser and the Flask application. This is the standard technique penetration tester uses to inspect exactly what a web application receives at the raw HTTP level, independent of what the client side Javascript of browser DevTools reports.

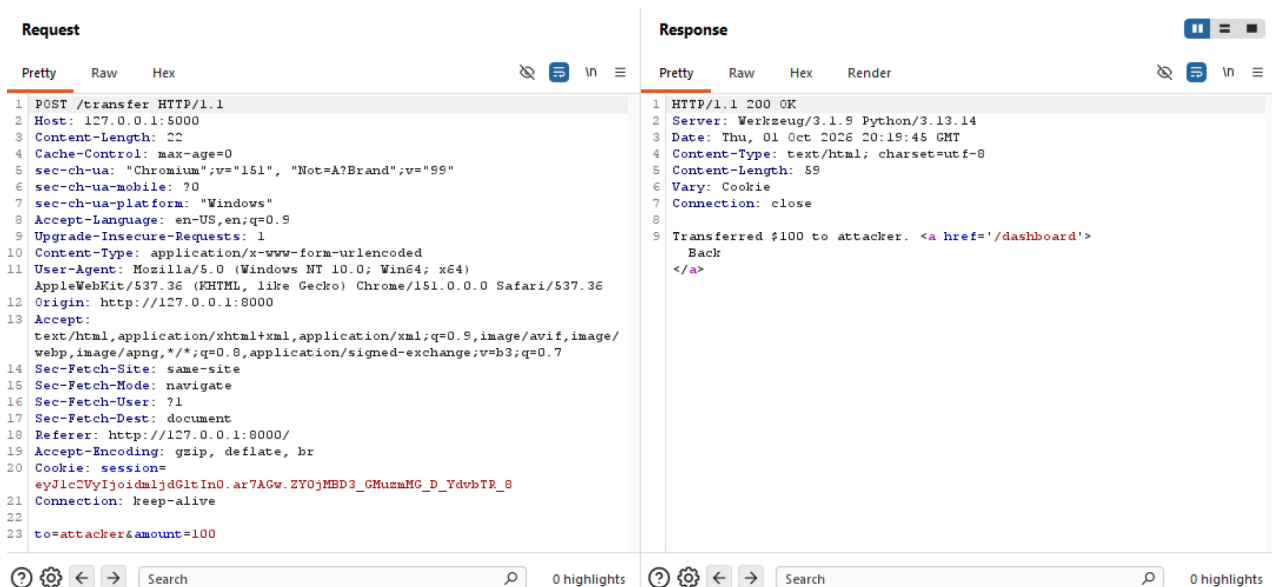
The browser was configured to route traffic through Burp's proxy (127.0.0.1:8000). The same baseline then attack sequence was repeated (first a legitimate and then the attack) through the proxy which is a legitimate transfer was first sent from the dashboard's own form and then the forged transfer was triggered via exploit.html. Both requests were captured in Proxy(Proxy History)

Intercept HTTP history WebSockets history Match and replace Proxy settings										
Filter settings: Hiding CSS and image content; hiding specific extensions										
#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title
1	http://127.0.0.1:5000	GET	/			200	250	HTML		
3	http://127.0.0.1:5000	GET	/login			200	489	HTML		
4	http://127.0.0.1:5000	POST	/login	✓		302	519	HTML		Redirecting...
5	http://127.0.0.1:5000	GET	/dashboard			200	591	HTML		
6	http://127.0.0.1:5000	POST	/transfer	✓		200	242	HTML		
7	http://127.0.0.1:5000	GET	/dashboard			200	590	HTML		
8	http://127.0.0.1:8000	GET	/			200	523	HTML		Directory listing for /
10	http://127.0.0.1:8000	GET	/exploit.html			200	1651	HTML	html	Win a Free iPhone!
12	http://127.0.0.1:5000	POST	/transfer	✓		200	247	HTML		

Proxy HTTP history



Capture of the legitimate request



Capture of the Forged (CSRF) Request

Interpretation for this is that the Burp capture shows at the raw HTTP level, the exact value of the cookie header sent on the forged request. It is byte for byte identical to session cookie used in the legitimate request while providing the browser attached the victim's real authentication cookie automatically, without the attacker needing to know, steal or guess it. The only distinguishing feature between the two requests is the Origin and Referer pair, which the server never checks. This is the clearest possible demonstration for the CSRF root cause where the application conflates "has a valid session cookie" with "the request is legitimate"

3.4.8 Summary

Three independent detection methods were used namely server side request logging, browser side DevTools inspection and proxy based interception capture. These all three converged on the same root cause *transfer* trusts any POST request that carries a valid session cookie with no verification of where that request actually originated. The runtime evidence demonstrates the vulnerability being actively exploited and its real impact.

3.5 Mitigation and Verification

3.5.1. Root cause recap.

/transfer root only authenticates the request by using the cookie of the session. Therefore, to mitigate the situation, require something that the attacker cannot obtain or supply as well as check from where the request came will be able to help.

For the mitigation, two layers were applied following a defence in depth approach.

1. Origin verification.

Before any state changing request is being processed, the server will always check the application's own address which is <http://127.0.0.1:5000>. Therefore, if there is any request that does not match with the origin, it will be rejected immediately with an HTTP 403 response.

```
app-mitigated.py
#block if the request is coming from any other place
if not origin.startswith(allowed):
    logging.warning(f"CSRF attempt blocked. Origin={origin}, Referer={request.headers.get('Referer','N/A')}")
    abort(403)
```

The allow list has chosen to give the access for the accepted origin as the server does not have to anticipate every possible attacker address.

2. CSRF token validation.

For the defence of depth, a random unique token generated per session through `secrets.token_hex(32)` and embedded in the form as a hidden field in every state changing form. When a request submits, the server checks the token that is stored in the server side session using the `secrets.compare_digest()` instead of just comparing the names with the simple comparison which is `actual_token=session_token` for the timing safe comparison. and the request will get rejected if the token is missing or does not match.

```
app-mitigated.py
#block if the sent and th actual token doesnt match
if not sent_token or not actual_token or not secrets.compare_digest(sent_token, actual_token):
    logging.warning(f"CSRF attempt blocked. Origin={origin}, Referer={request.headers.get('Referer','N/A')}")
    abort(403)
```

If the attacker gets the chance to bypass layer one of security, layer 2 will be able mitigate it using the comparison stated above because the token will not be received by the attacker as it cannot be read or guessed.

3.5.2. Before and After verification

Both `app.py` and `app-mitigated.py` were run with the same unmodified `exploit.html` to observe the attack and the clear before and after comparison is mentioned below.

Before mitigation

Balance before the attack:

Your balance: \$1000

When the attack happens:

Transferred \$100 to attacker. [Back](#)

Balance after the attack:

Your balance: \$900

The attacker's forged request was processed identically to a legitimate one. The only distinguishing field was the origin or else Referer which means the original application was never checked as well as the transfer completed which moved the funds from victim to attacker.

After mitigation

Balance before the attack:

Your balance: \$1000

When the attack happens:

Forbidden

You don't have the permission to access the requested resource. It is either read-protected or not readable by the server.

Balance before the attack:

Your balance: \$1000

Running the unmodified same exploit produced a blocked attempt which is http 403 error. The forged request has been rejected at the initial phase which is at origin check even before the token check or the fund transfer. Therefore, no funds have moved.

3.5.3 Confirming the fix does not break legitimate use

To confirm that the mitigation does not introduce a regression, a legitimate transfer was performed.

Legitimate transfer before:

Admin - All Account Balances	
Account	Balance
victim	\$1000
attacker	\$0
user	\$500

Legitimate transfer:

Transferred \$300 to user. [Back](#)

Legitimate transfer after:

Admin - All Account Balances	
Account	Balance
victim	\$700
attacker	\$0
user	\$800

The legitimate transfer has been succeeded, the token was validated correctly as well as balance has been updated in the expected way which confirms that the mitigation was able to block the forged request without breaking the normal functionality. A 403 error code was displayed as the output whenever the origin or the token does not match with the expected results.

3.5.4. Why this mitigation is useful

CSRF is possible whenever a state changing endpoint trusts the request as it will session the unique session cookies embedded such as a session cookie. The browser attaches the session cookie without even checking who sent the request. Therefore by requiring a matching origin as well as a secret token that the attacker cannot read will be able to mitigate the CSRF attack by removing two main parts that the exploit relies on which means being trusted by default as well as having no server side to prove that

the request was being made by the original user(victim). The origin check helps to stop the majority of forged cross origin requests cheaply, while the token check will be able to provide redundancy if the origin check is ever bypassed which means the defence depth rather than relying on a single mitigation method.

4. Vulnerability 2: Log Injection

4.1 Explanation

The log file is considered as a significant record as it is known as trustworthy because organizations rely on it for monitoring, accountability as well as forensic investigation after an incident. The value of the log file is that it can be trusted what happened in the entry points which includes the failed as well as successful logins. Log injection is the attack which breaks the core purpose of the log file. Therefore, if an attacker will be able to forge or alter the entries the logs will no longer be considered as trusted as evidence.

Log injection, which is also known as CWE-117: Improper Output Neutralisation for logs, occurs when the application does not clean the data that was entered by the user in the user-controllable section. The log file contains the log details of the application, which include usernames, search terms or request headers to record evidence of the activity. The newline character is being used to separate one entry from another since the data is being recorded and stored as plain text. Therefore, attackers take advantage of this criterion to abuse the vulnerability.

The flaw arises when untrusted input is logged verbatim. Log injection is a vulnerability where the attacker manipulates user-controllable data that the application records in the log files. As an example, when the attacker uses newline character or URL-encoded as %0d and %0a , these will mislead the information of the log file as it can lead to a current line cut off and start a line. Hence, the application keeps writing the attacker's input into the log file as if it were a system-generated or legitimate entry.

Log injection can provide attackers with multiple advantages. Such as, log entries can be manipulated by attackers and create inaccurate event details including successful administration logins even though no admin logins were done or to create an innocent user appear responsible for an action. Attackers will be able to flood the logs with inaccurate blurry evidence of their real activity through the log injection attack by making any further investigations. Additionally, in some long injection cases, Cross site attacks (XSS) will be released for those who investigate the logs if the logs displayed through a web interface without the proper encoding which includes HTML or JavaScript. Log injection will be able to reduce the reliability, the log integrity and frustrating investigations of logs by making it hard for security teams to detect and investigate malicious activity.

4.2 Vulnerable Code

The vulnerability 2, which is open to log injection, lies in the /login route. Whenever a user attempts to log in, it writes to the log file using the user-supplied username directly without even neutralising it, even though the login fails or passes. Therefore, no sanitisation of the newline or carriage return characters has been implemented.

The exact vulnerable code is attached below.

```
app.py

#this is the VULNERABILITY- because this allows the raw username written to the log without sanitizing.
logging.warning("Failed login for user: %s",username)
#css for the failed login
return """

<link rel="stylesheet" href="/static/style.css">
<div style="text-align:center; margin-top:60px;">
  <h2>Invalid credentials</h2>
  <p>Please try again.</p>
</div>
"""

# only run this if the file is executed directly
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

To view the full document, visit the GitHub repository or Appendix B. Since the attacker will be able to control the username and it is being written into the log file, the application will not be able to prevent the attacker from embedding the newline characters in the username. A log file contains one entry per line. Therefore, an injected new line ends the new line's entry early as well as starts a new line that the attacker wishes to write. The application will not be able to determine which lines are the genuine log entry from a forged one written in the provided way. Finally, this is the flow in which an exploitation can happen (CWE-117: Improper Output Neutralisation for Logs).

4.3 Exploitation

The exploit was made using python which is exploit.py which sends the two requests to the endpoint of the vulnerable login. The first request will be the genuine incorrect entry which produces a normal failed login. The second request will be the exploitation which carries a crafted username which contains a newline followed by a fake log line. Since the application logs does not sanitize the usernames, the attacker's crafted log line will be written as a separate as well as legitimate entry.

The attack code is stated below,

```
exploit.py

#The attack (fake log line)
# attack with the current time
now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
payload = f"attacker\r\n{now} INFO Successful login for user: admin"
#sends the malicious login attempt
helper({"username":payload, "password":"attacker"})

print("Attack sent.")
```

The attack can be made much stronger by forging timestamps by a few seconds or minutes from the current time which helps to make the investigation much harder to detect. For the demonstration purposes, a forged entry uses the current time as the real entry.

How the attack works step by step.

1. The attacker will use the exploit.py file against the vulnerable application's endpoint which is /login.
2. A normal incorrect login will be sent as the first log which produces the genuine log entry for the comparison.
3. The script will retrieve the current date as well as time and create a username which contains the carriage return and newline which represents with `/r/n` followed by a log line written in the exact time in the application.
4. When the username is submitted as the failing login the vulnerable logging.warning line will be reached. The application will write the raw username without sanitizing into the app.log which contains all the information of the entries. The embedded new line (`\n`) will separate the genuine line from the attack line as it is placed at the end of the real entry. Therefore, the fake log line will have a new line.
5. Since the forged line uses the current timestamp, it acts as legitimate from a genuine "Successful login for user: admin" entry even though there was not an actual entry.

A successful log injection will allow the attacker to forge the log entries, plant false evidence as well as frame a fake user that never existed as a genuine user. Therefore, the same technique can be used to blur the evidence of the real malicious activities which helps the attacker to obstruct the forensic investigations. More broadly, this vulnerability undermines the integrity as well as trustworthiness of the whole audit trail since the logs are primary record defenders which rely on the detect as well as reconstruct the attack.

4.4 Detection and Tracing

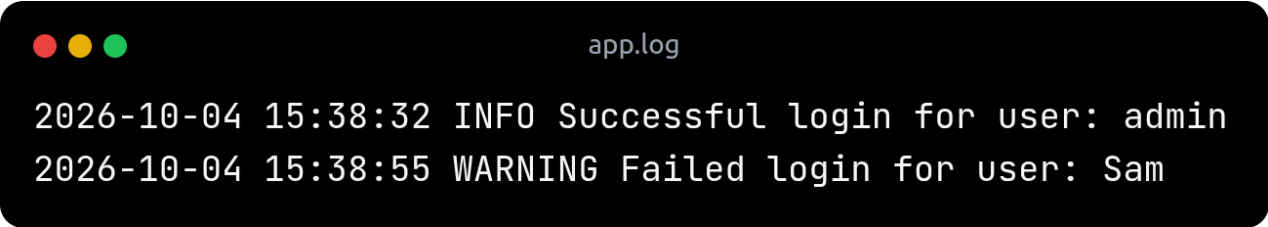
4.4.1 Approach

Detection for log injection involves inspecting the application's log file, since this is the artifact the attacker directly targets. Unlike CSRF, network interception tools such as Burp Suite or OWASP ZAP cannot meaningfully detect log injection. At the HTTP level,

the malicious request appears as an ordinary failed login and the injected content appears only as a value in the username field. The attack targets exclusively server-side at the moment the unsanitised username is written into the log, so the log file itself is the correct and only reliable detection surface.

4.4.2 Baseline - legitimate log entries

To demonstrate what a normal logging attempt looks like, ordinary logins were performed first. Each produces exactly one log line.



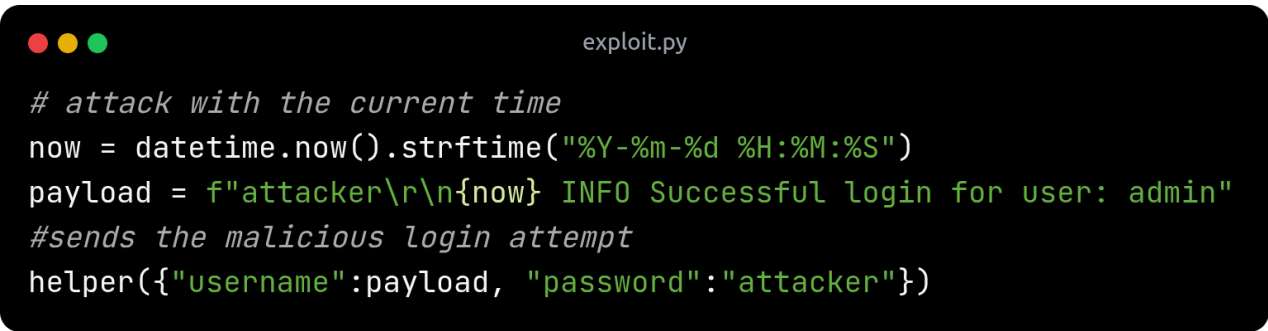
```
app.log

2026-10-04 15:38:32 INFO Successful login for user: admin
2026-10-04 15:38:55 WARNING Failed login for user: Sam
```

As shown above, each login attempt, successful or failed, produces a single, well formed entry in the format *timestamp LEVEL message*. One request maps to one log line. This is the baseline the attack is compared against.

4.4.3 Attack - forged log entry via injection

The exploit ([exploit.py](#)) then submitted a login with a crafted username containing a carriage-return/newline sequence (`/r/n`) followed by a fake log line.



```
exploit.py

# attack with the current time
now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
payload = f"attacker\r\n{now} INFO Successful login for user: admin"
#sends the malicious login attempt
helper({"username":payload, "password":"attacker"})
```

This produced the following in app.log.



```
app.log

2026-10-04 15:38:32 INFO Successful login for user: admin
2026-10-04 15:38:55 WARNING Failed login for user: Sam
2026-10-04 16:05:35 WARNING Failed login for user: Siluni
2026-10-04 16:05:35 WARNING Failed login for user: attacker
2026-10-04 16:05:35 INFO Successful login for user: admin
```

This is the evidence of the attack. A single failed login request produced two log lines (last two lines in the image). The 4th entry is the genuine entry recording the failed attempt. The 5th entry is entirely forged. It was never a real login made by a person, it exists only because the attacker's `\r\n` broke out of the intended log line and injected a

fabricated entry of their own. Several tells prove this line is forged rather than the real one.

4.4.4 Detection - identifying the forged entry

The forged entry is identical in format to a legitimate one, so it cannot be identified by its text alone. It is instead exposed by several anomalies, each of which independently indicates tampering.

1. Byte-level carriage return (\r)

Inspecting the raw bytes of the log with `cat -A` or `grep` reveals a carriage return character embedded in the injected line, which legitimate entries never contain.

```
→ vulnerable git:(main) x bash
sam@Sams-PC:~/Desktop/ISEC3004-Security-Project/vuln2-log-injection/vulnerable$ cat -A app.log
2026-10-04 15:38:32 INFO Successful login for user: admin$
2026-10-04 15:38:55 WARNING Failed login for user: Sam$
2026-10-04 16:05:35 WARNING Failed login for user: Siluni$
2026-10-04 16:05:35 WARNING Failed login for user: attacker^M$
2026-10-04 16:05:35 INFO Successful login for user: admin$
```

`cat -A app.log`

```
→ vulnerable git:(main) x grep -P '\r' app.log | sed 's/\r/[R]/g'
2026-10-04 16:05:35 WARNING Failed login for user: attacker[R]
```

`grep -P '\r' app.log | sed 's/\r/[R]/g'`

In the developed application, a legitimate log entry only ends with a newline (`\n`). The presence of a `\r` or any other unusual character inside a line is direct evidence of outside tampering with the log file.

2. One request produced two entries

A single login request can only generate one log line (as shown in 4.4.2 / 4.4.3). The attack produced two, which is structurally impossible for a login and signals that extra content was injected.

3. Identical timestamps

The forged line shares the exact timestamp of the genuine failed login line above it. Two genuinely separate events would rarely share an identical timestamp. This can be easily fixed by the attacker by editing the current time by seconds, but in this application this is somewhat strong evidence that something suspicious has happened to the log file.

4. Timestamp mismatch with different timezones

In this application, the attacker and the server run on the same machine, so the injected timestamp that is generated by the exploit's `datetime.now()`, matches the server's clock. However, if an attacker operating from a different timezone would inject a timestamp reflecting their own local clock, it will lead to timestamp inconsistency.

4.4.5 Real world considerations

All the detection methods discussed in 4.4.4, can be easily evaded by a sophisticated attacker. Injecting with only a `\n` without `\r`, would remove the first detection method. Matching the server's timezone and offsetting the timestamp would defeat timing tells. A log injection is an exploit that is harder to land for an attacker with limited access to the code they are attacking, making the exploit less likely. However, it

doesn't apply to open-source projects or if the attacker has access to the files. So, instead of guessing if an attack has happened or not, the best thing is sanitising input before it is logged (Section 4.5).

4.4.6 Summary

The exploitation was identified through four independent indicators, an embedded carriage return byte, a single request producing two entries, identical timestamps and timestamp mismatch with timezones. This directly demonstrates the impact of the vulnerability, the system's record of what happened is compromised and it now cannot be trusted.

4.5 Mitigation and Verification

4.5.1 Mitigation technique

The vulnerability existed because the raw username field from the login form was passed directly into `logging.warning()` / `logging.info()` with no validation or sanitisation allowing an attacker to embed a carriage return or line feed (`\r\n`) sequence followed by a fabricated log line marking the application's own log writer produce a forged entry.

Two layers of mitigation were applied, following a defence in depth approach.

1. Input validation (Primary control)

This is an allow list regular expression that restricts the username to a safe, expected character set (letters, digits, dot, underscore, hyphen, 1-32 characters) before it is used anywhere in the request handler.

```
app.py
USERNAME_PATTERN = re.compile(r"^[a-zA-Z0-9._-]{1,32}$")
```

Input validation

Any request whose username does not match this pattern is rejected immediately with an HTTP 400 response and a fixed safe message is logged instead of the attacker's raw input.

```
app.py
#Mitigation - reject anything outside the expected username format before it is used anywhere including logging.
if not USERNAME_PATTERN.match(username):
    logging.warning("failed login attempt with invalid username format (Input rejected)")
    return ""

<link rel= "stylesheet" href="/static/style.css">
<div style="text-align:center; margin-top:60px;">
    <h2>Invalid credentials</h2>
    <p>Please try again.</p>
</div>
""" , 400
```

An allow list was chosen over a block list (e.g. specifically stripping `\r` and `\n`) as the primary control because it fails closed. It rejects anything unexpected by default, rather than relying on the developer to anticipate every dangerous character.

2. Output sanitisation at the log call (secondary call)

As a defence in depth a `sanitize_for_log()` helper strips any control characters (CR, LF and other non printable bytes) from a value immediately before it is written to the log even though validation above should already guarantee the value is clean by this point.

```
def sanitize_for_log(value: str) → str:

    """Sanitize a string for safe logging."""
    # Mitigation 02 : Sanitization
    # This replaces any CR or LF characters with a safe placeholder. This prevents log injection attacks.

    return value.replace("\r", "\\r").replace("\n", "\\n")
```

```
#mitigation - sanitize before logging
logging.warning("Failed login for user: %s", sanitize_for_log(username))
#css for the failed login
return ""
```

```
logging.info("Successful login for user: %s", sanitize_for_log(username))
```

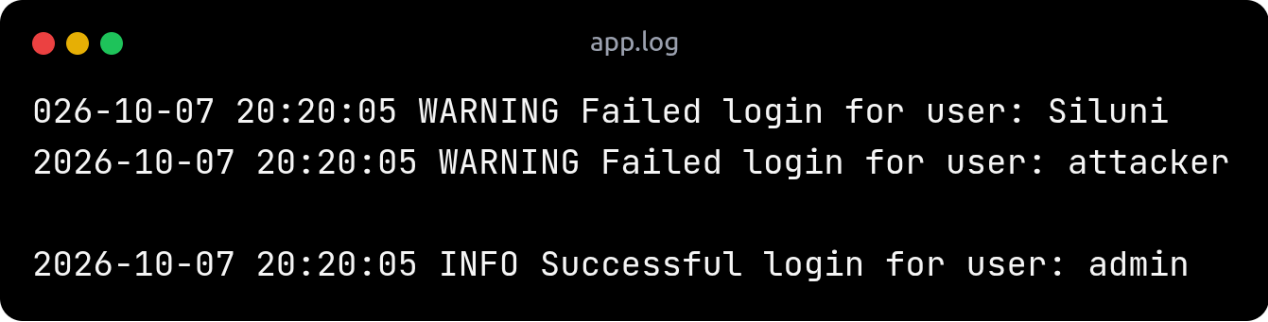
Input Sanitisation

This second layer ensures that even if a future code change introduced another part to the logger that forgot to call the validator, a forged log entry would still not be possible since the ability to inject a literal newline is the mechanism the entire attack depends on.

4.5.2 Before and After Verification

The exact same unmodified [exploit.py](#) script was run against both the original vulnerable [app.py](#) and the mitigated [app.py](#) with no changes to the exploit itself to give the before and after comparisons clearly.

Before mitigation - app.log

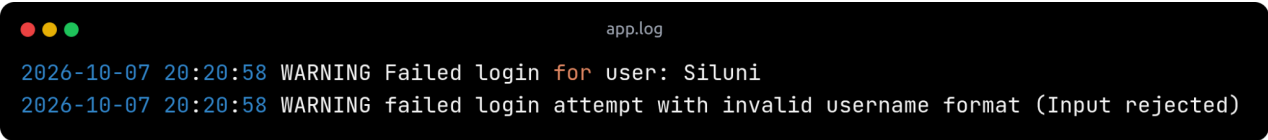


```
app.log
026-10-07 20:20:05 WARNING Failed login for user: Siluni
2026-10-07 20:20:05 WARNING Failed login for user: attacker

2026-10-07 20:20:05 INFO Successful login for user: admin
```

The second and the third lines above are both the product of a single malicious request. The attacker's payload broke out of the intending log line using `\r\n` and injected a fabricated *INFO Successful login for user : admin* Entry which is a forged success event that never actually happened.

After mitigation - app.log

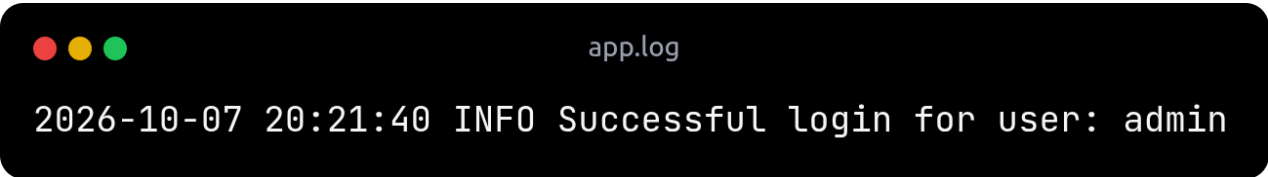


```
app.log
2026-10-07 20:20:58 WARNING Failed login for user: Siluni
2026-10-07 20:20:58 WARNING failed login attempt with invalid username format (Input rejected)
```

Running the identical exploit script against the mitigated application produces only a single safe log line. The malicious username never appears in the log at all where the request was rejected at the validation stage before it could reach the logger and the forged admin success entry no longer appears.

4.5.3 Confirming the fix does not break legitimate use

To confirm the mitigation does not introduce a regression a legitimate login (admin) was also tested against the mitigated application



```
app.log
2026-10-07 20:21:40 INFO Successful login for user: admin
```

The legitimate login succeeds (HTTP 200) and is logged correctly and safely confirming the fix blocks the attack without breaking normal functionality where a HTTP 400 response is returned only for input that fails the username format check.

4.5.4 Why this mitigation is effective

Log injection is possible whenever untrusted input reaches a logging sink without first removing or escaping the characters that give that sink structural meaning which is a plain text line based log that character is the new line. By rejecting any username containing a new line (any other unexpected character) before it is ever processed the mitigation removes the attacker's only mechanism for injecting additional fabricated lines into the log. The secondary sanitisation layer provides redundancy in case a

future code path bypasses the primary validation, which is a recommended defence in depth practice rather than relying on a single control.

5. Mitigation Strategies and Security Recommendations

Although the two vulnerabilities investigated in this project namely Cross Site REquest Forgery and Log Injection belong to different classes of flaw (a trust session handling weakness versus and unsanitised input weakness) both were fixed using the same underlying security principle which is never trust data that originates outside the server's own control and verify or reject it before it is used for anything consequential. For CSRF the "data" was the session cookie, which the server wrongly treated as sufficient proof that a request was legitimate. For Log Injection the "data" was the username field which the server wrote directly into a security log without first checking its shape. In both cases the root cause was the same which is a trust boundary existed between untrusted input and a sensitive sink and no validation was applied at that boundary.

This leads to a set of broader recommendations that extend beyond the two specific fixes implemented in the mitigation sections.

Validate and sanitise at every trust boundary not just the obvious one. The log injection vulnerability was particularly instructive because the sink being attacked was the application's own security logging where the very mechanism meant to detect malicious activity. A development team should treat logging statements as a trust boundary in their own right, not merely an internal implementation detail since log integrity is frequently relied upon during incident response.

Prefer allow lists over block lists wherever the valid input space is well defined. The log injection mitigation validated usernames against a fixed set of permitted characters rather than trying to strip or escape a list of "dangerous" characters. An allow list fails closed where anything unexpected is rejected by default, whereas a block list fails open against any character the developer did not think to exclude.

Apply defence in depth rather than relying on a single control. Both mitigations in this report used more than one layer namely the log injection fix combined an anti CSRF token with restored samesite cookie defaults. If one layer is bypassed by a future code change or an un anticipated edge case, the second layer still holds.

Rely on well tested frameworks and libraries rather than rolled security logic. Flask WTF's CSRFProtect was chosen over a custom token implementation precisely because token generation, storage and comparison are easy to get subtly wrong (e.g. non constant time comparison, predictable token generation). Many modern web frameworks now provide CSRF protection by default and disabling it which the vulnerable version of this project's banking app effectively did which should be treated as a deliberate, reviewed decision rather than a default state.

Treat Security relevant configuration (debug mode, hardcoded secrets, cookie flags) as part of the attack surface even when it is unrelated to the vulnerability being demonstrated. The static analysis in section 3.4.7 incidentally flagged debug = True and a hardcoded secret_key alongside the CSRF finding this neither was part of tis project's chosen vulnerabilities but both represent the same class of risk which is a control that exists but was left disabled or weakened and a real development team's

code review process should catch these regardless of what specific vulnerability a given sprint focused on.

Use detection tooling that operates at more than one stage of the pipeline. This project deliberately used three different categories of detection namely runtime application logging during exploitation, network level interception (Burp suit) at the transportation layer and browser based detection, because each catches different failure modes. A team relying on only one of these would have no way to detect an attack actually succeeding against production, even if the code had been reviewed for this flaw beforehand.

6. Evidence of Project Management

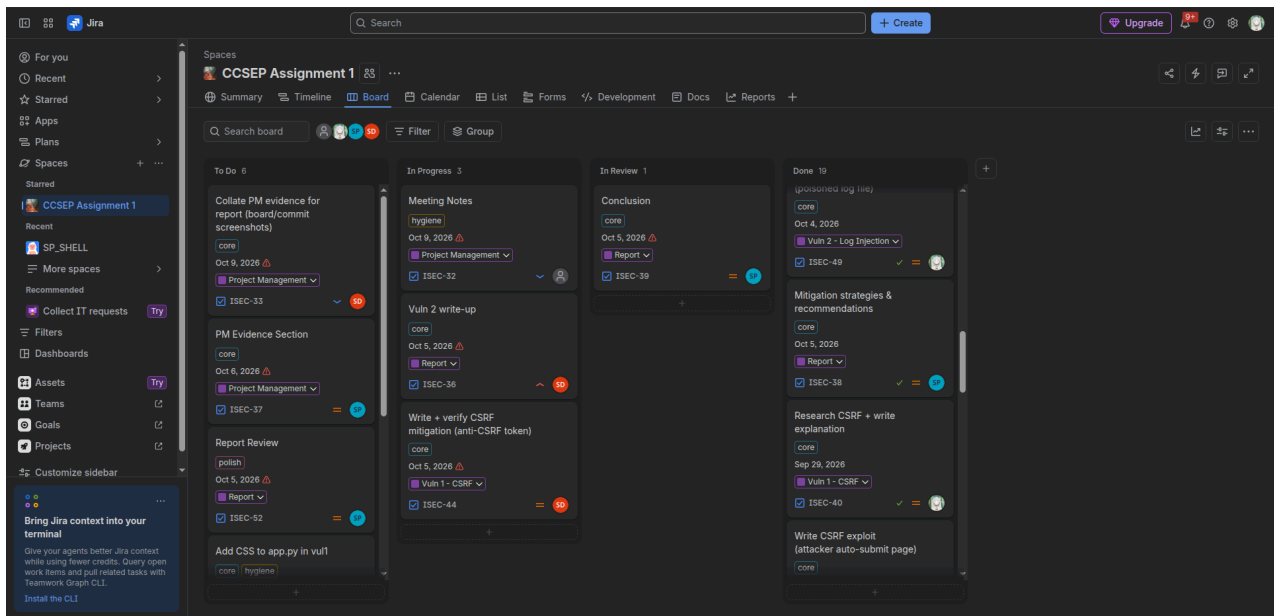
This section provides evidence of how the project was coordinated using the practices set out in section 2. Key evidence is shown below with short captions and larger records (full commit history, board exports and complete meeting notes) are included in the appendices and referred where relevant.

6.1 Project artifacts and links

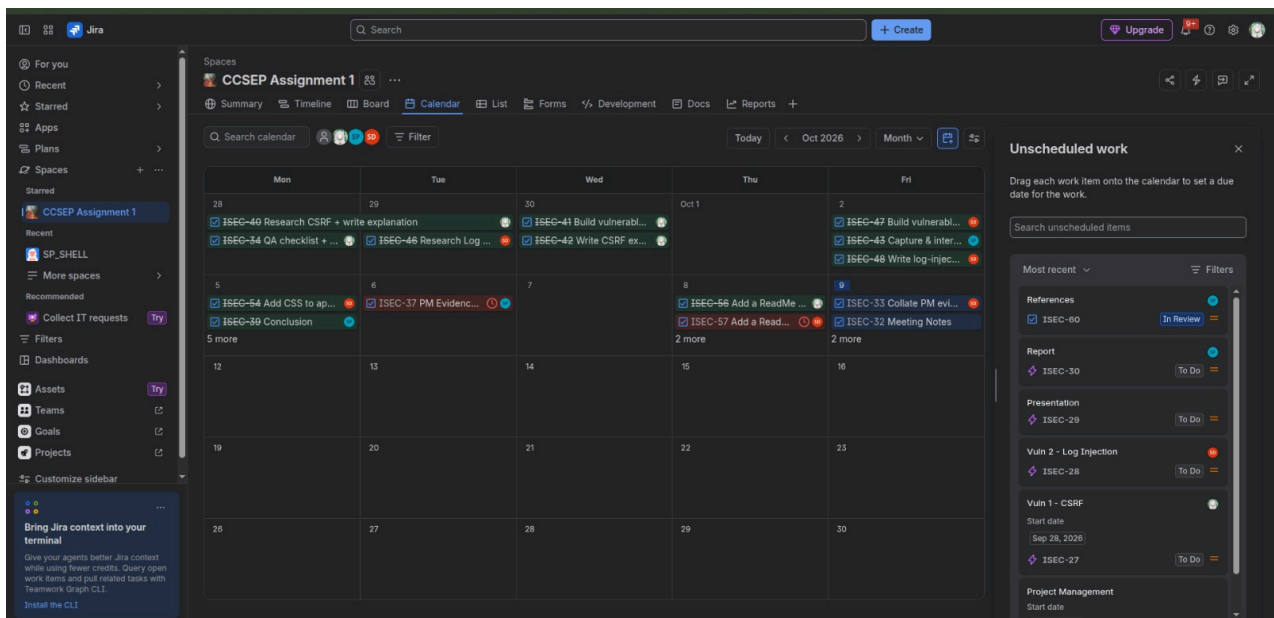
Artefact	Tool	Purpose	Link
Task board	Jira	Task allocation, Progress tracking	JIRA BOARD
Source Code	Github	Version Control, branches, peer review	GIT REPO
Documents	Google Drive	Meeting notes, report drafts, shared documents	ISEC3004 Docs
communication	Whatsapp	Day to day Coordination and scheduling	

6.2 Task allocation and Jira Board

Tasks were created on the jira board before work began and assigned to named team members according to the responsibilities in section 2.1. Each card was moved through the columns To Do, In Progress, InReview and Done as work advanced. Please see the [Appendix B](#) for more details.



Task Assignment in Jira



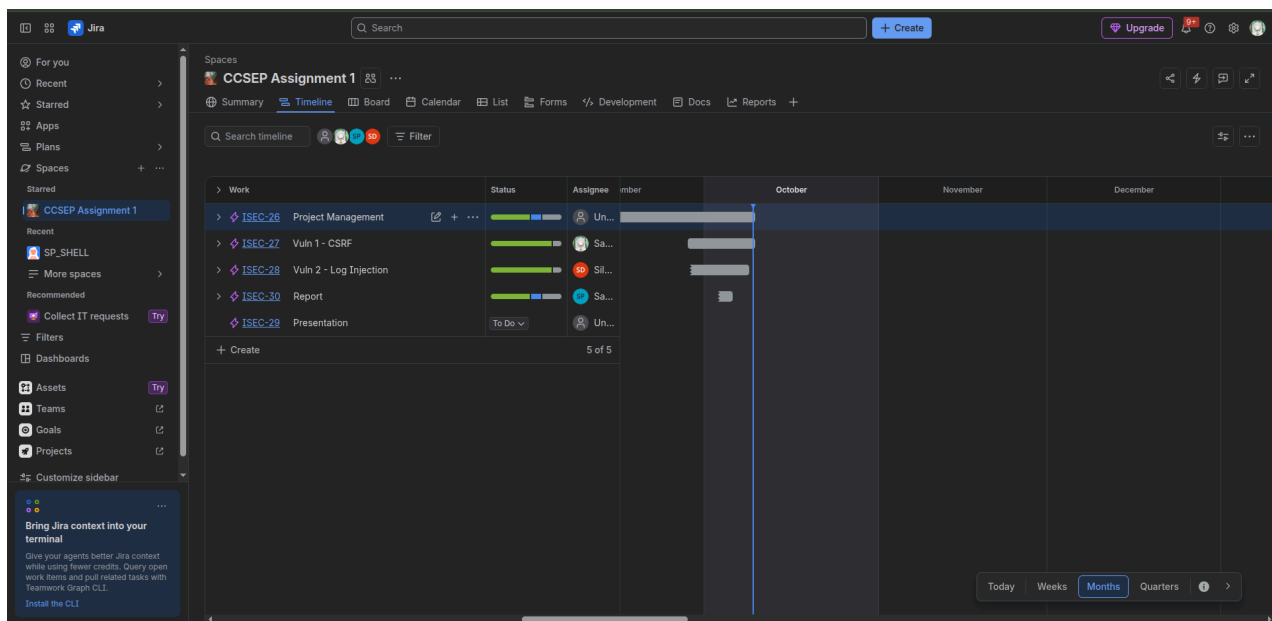
Calendar view with tasks and assignee

6.3 Schedule and milestones

The project was planned around the milestone schedule set out in section 2.2, anchored by a mid project check point at which both vulnerable applications and their exploits were required to be working. This ensured problems surfaced with enough time to recover before the submission deadline. Detailed per task due dates were maintained on the Jira board. [Appendix B](#)

Date	Planned Milestone	Status
15 Sep	Group formed, Vulnerabilities chosen, tools and roles set	Done
19 Sep	GitHub and Jira Board setup, starter tasks assigned	Done

2 Oct	Halfway gate , both vulnerable programs built, exploits working	Done
4 Oct	Detection evidence captured, mitigations implemented and verified	Done
6 Oct	Full report draft complete, internal reviews begin	Done
9 Oct	Report and Code submission	Done
Week 12	Presentation	Upcoming



Jira Time line

6.4 Version Control

All code was developed in a shared Github repository with each member committing their own work so individual contributions are traceable. Work was separated into branches by purpose so that detection and mitigation work could be reviewed independently before merging. [Appendix C](#)

Add CSRF mitigation	siluni-deshapriya committed 2 days ago	f1317fe		
CSRF mitigated application	siluni-deshapriya committed 2 days ago	08c1bdf		
Commits on Oct 7, 2026				
replace transfer.log to correct location	Sammm2006 committed 2 days ago	8e5985b		
Merge pull request #3 from siluni-deshapriya/mitigation-Vul2	Sammm2006 authored 2 days ago	Verified bb34d5f		
Mitigation code done for Log Injection	Santhosh19420 committed 2 days ago	87cc569		
Commits on Oct 3, 2026				
fixed issue #1. Log files saves to CSRF folder	Sammm2006 committed last week	0e64072		
exploitation code for log injection vulnerable code	siluni-deshapriya committed last week	6ea5a3e		
Merge pull request #1 from siluni-deshapriya/detection	Sammm2006 authored last week	Verified e767c5f		
Vulnerable code for the log injection (vulnerable 2) - commented version	siluni-deshapriya committed last week	118ebe1		
Vulnerable code for the log injection (vulnerable 2)	siluni-deshapriya committed last week	b26cde3		
Commits on Oct 2, 2026				
Code was updated for detection logging and transfer.log is attached	Santhosh19420 committed last week	0c90c52		

Git Hub commit History

Branches						New branch
Overview Yours Active Stale All						
Q Search branches...						
Default						
Branch	Updated	Check status	Behind	Ahead	Pull request	
main	2 minutes ago			Default		...
Your branches						
Branch	Updated	Check status	Behind	Ahead	Pull request	
mitigation-Vul2	2 days ago		8	0	#3	...
detection	last week		14	0	#1	...
Active branches						
Branch	Updated	Check status	Behind	Ahead	Pull request	
mitigation-Vul2	2 days ago		8	0	#3	...
detection	last week		14	0	#1	...

Branch list

6.5 Effort Estimation

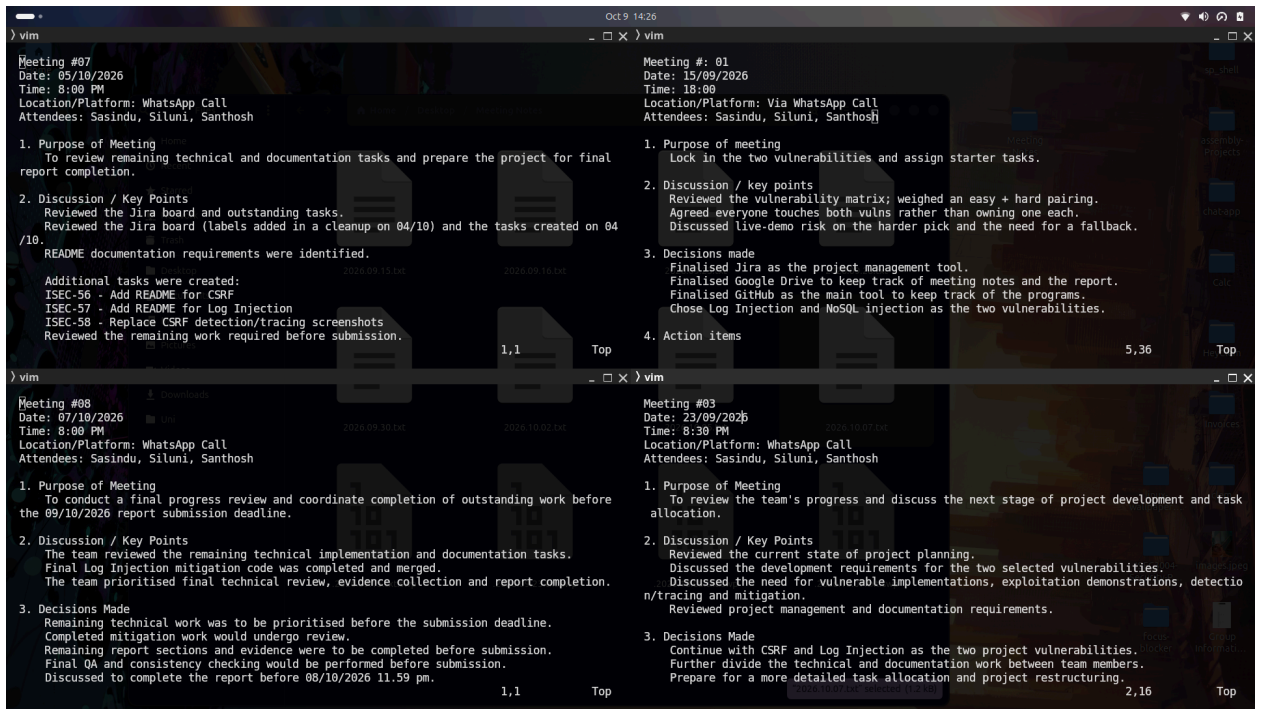
Before work began major tasks were estimated in hours on the Jira board so the team could gauge the total workload and balance it fairly. The technical pipeline for each vulnerability was estimated at approximately 18 hours, with the overall workload

totalling around 55 hours. The full breakdown is given in section 2.4. Please see Appendix for more detailed Information. [Appendix E](#)

Member	Estimated Hours
Sasindu	18 hours
Siluni	19 hours
Santhosh	18 hours
Total	~55 hours

6.6 Meeting Records

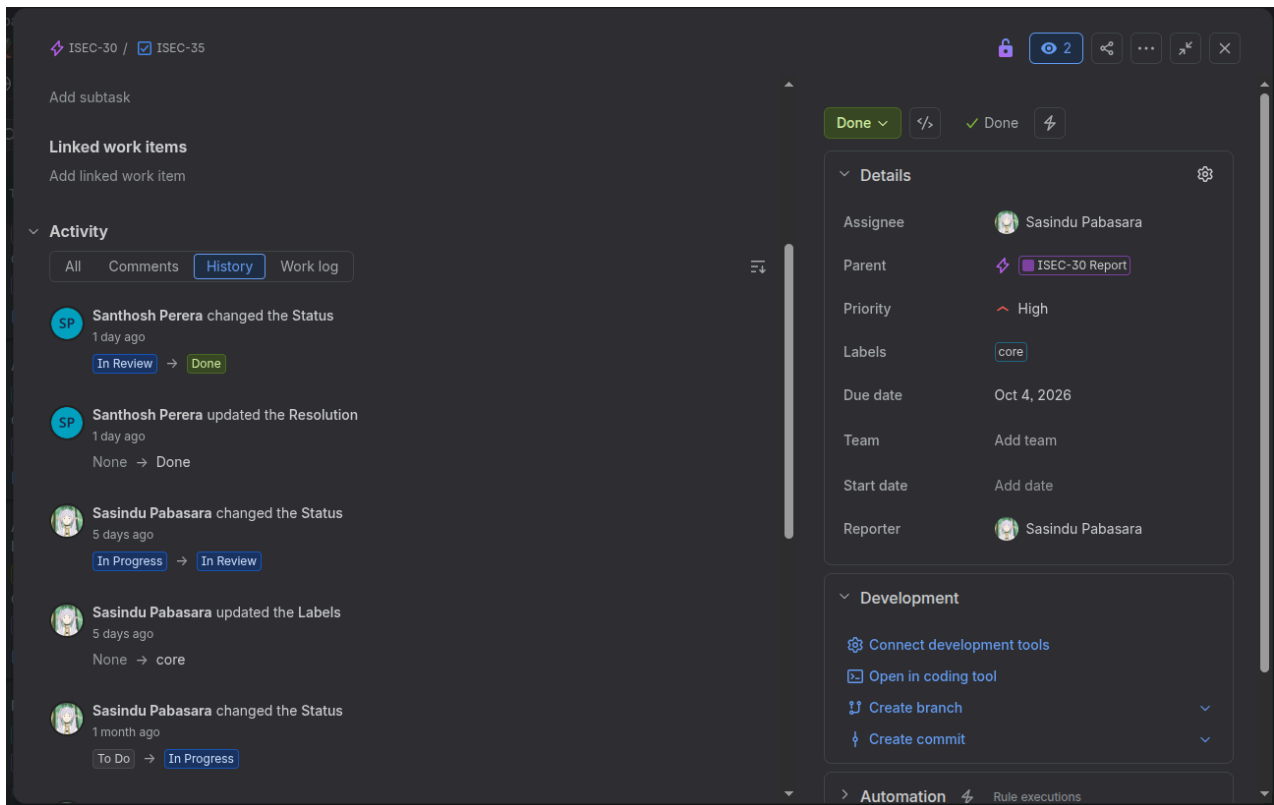
The team met regularly throughout the project to make decisions, assign work and review progress. Each meeting was documented as a short summary recording the date, decisions made and action items maintained in the group’s Google Drive. Please refer to [Appendix D](#)



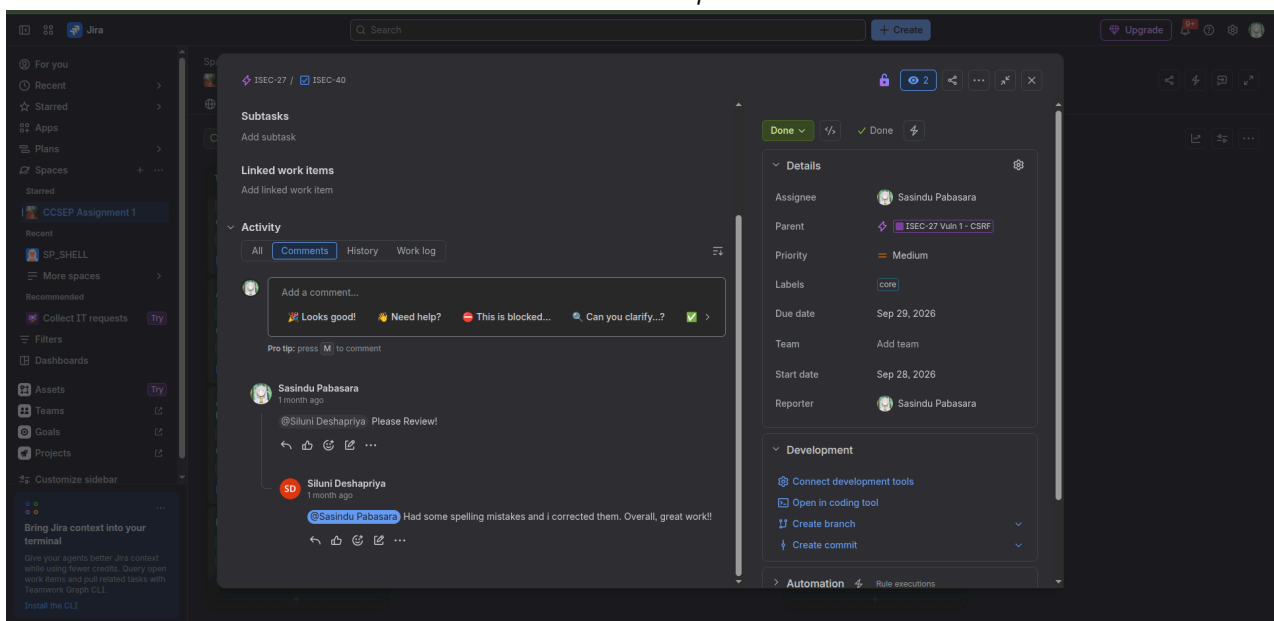
Meeting Notes

6.7 Quality Assurance

Quality assurance was built into the workflow rather than left until the end. Code was peer reviewed through GitHub before merged, each exploit was independently validated by a second member to confirm it reproduced reliability and reach mitigation was verified by re-running the original exploit against the fixed code to confirm the attack was blocked. Because two builders each validated the other’s vulnerability, quality checks were consistently carried out by someone other than the original author. These checks were tracked through the “In Review” stage of the Jira board before any task was marked complete. [Appendix F](#)



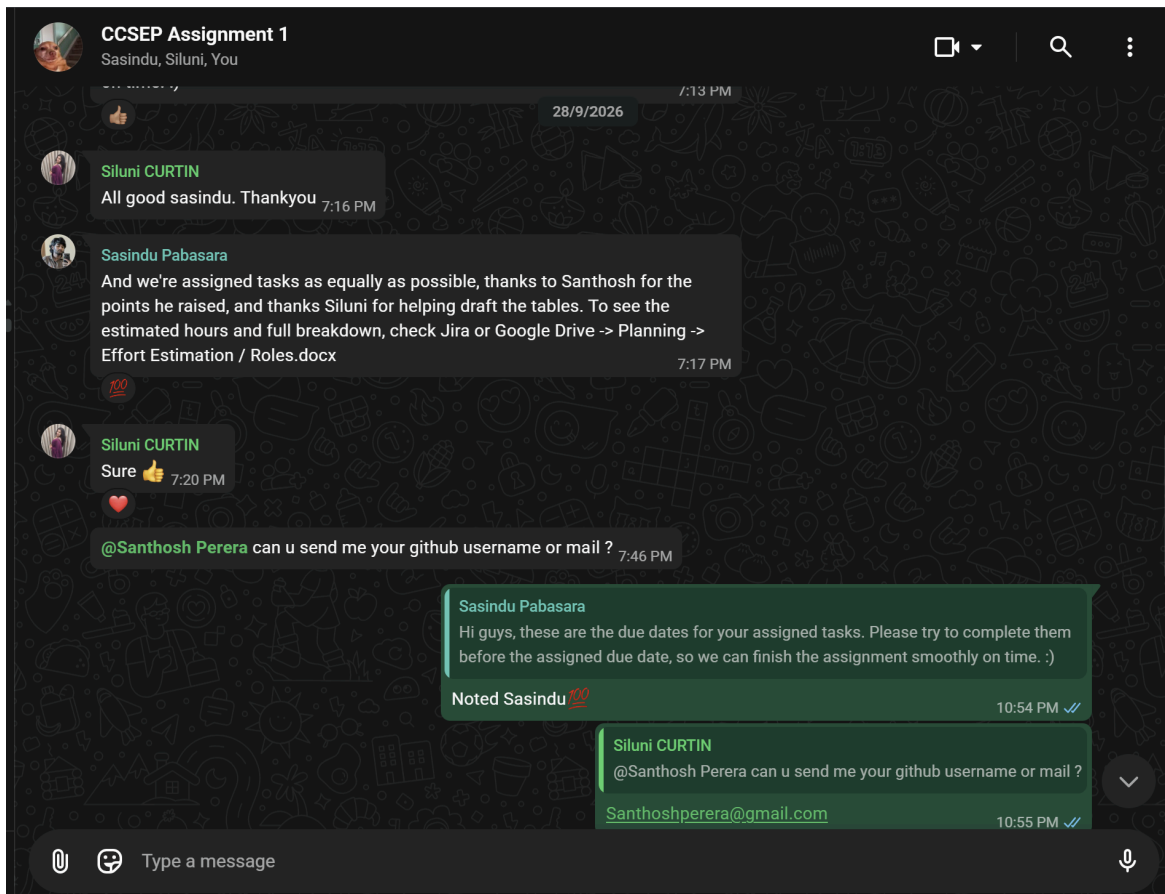
Task review and completion



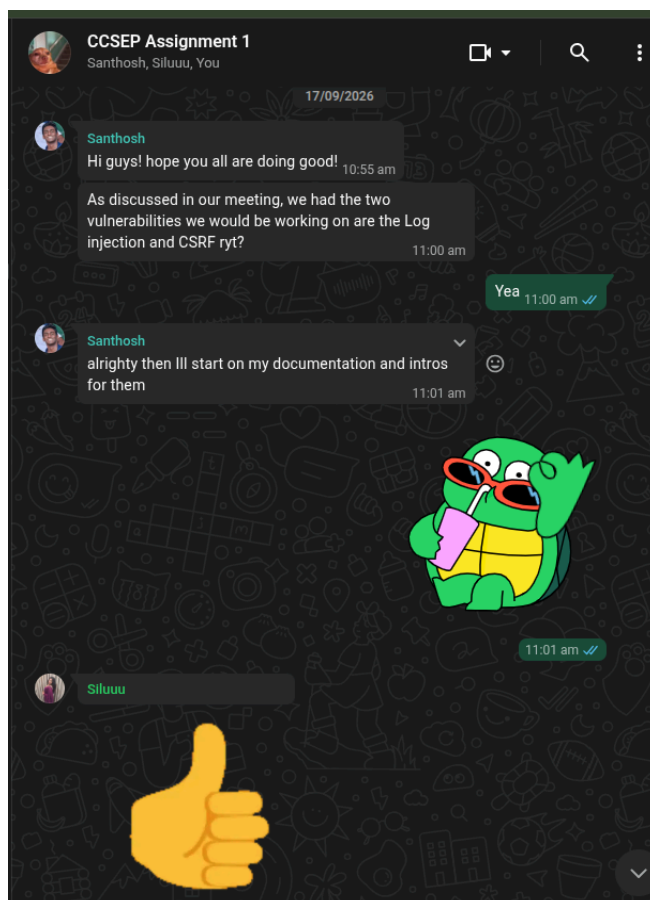
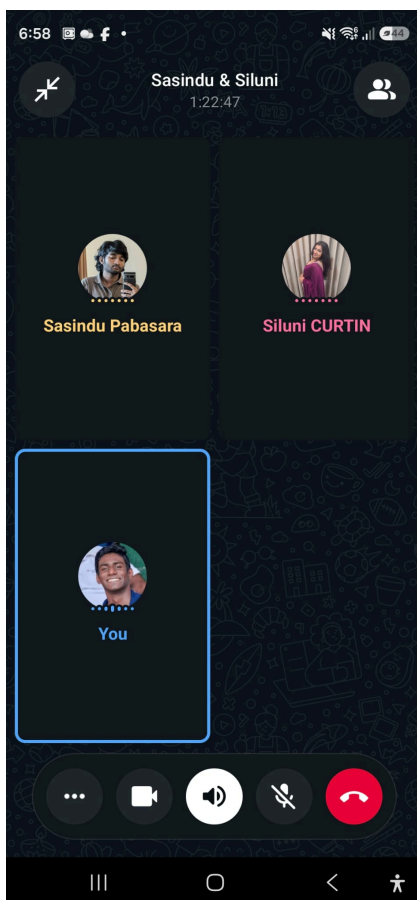
Code review comment in JIRA

6.8 Communication

The group used whatsapp as the communication channel for our daily conversations and doubt clarifications.



Whatsapp Group chat



Group Discussions

7. Conclusion

This Project investigated two vulnerabilities namely Cross Site Request Forgery and Log Injection, across the full lifecycle required by the assignment which is building a deliberately vulnerable application, exploiting it, detecting and tracing the resulting malicious activity using multiple independent tools, implementing a mitigation and verifying that mitigation against the original exploit.

For CSRF, the group demonstrated that a session cookie only authenticated check on a fund transfer endpoint allowed an attacker to forge a state changing request from an entirely separate origin, resulting in real financial impact in the demo application. Detection was carried out through server side request logging, chrome DevTools inspection and interception proxy capture via Burp suite, with all three methods independently confirming the same root cause. The mitigation where the Flask WTF's CSRFProtect combined with restored SameSite cookie defaults was verified to block the identical attack while leaving legitimate application functionality unaffected.

For Log Injection, the group demonstrated that writing unvalidated user input directly into a security log allowed an attacker to forge additional, fabricated log entries, undermining the very mechanism intended to provide an audit trail of login activity. The mitigation was to allow list input validation backed by a secondary log sanitisation layer which verified against the unmodified exploit script, successfully rejecting the malicious payload while preserving normal login functionality.

Across both vulnerabilities, the project reinforced a common lesson which is security failures in these two cases did not stem from exotic attack techniques, but from a missing validation step at a clearly identifiable trust boundary. This is consistent with both vulnerabilities' continued prevalence in real world applications and underscores why input validation, defence in depth and the use of established security libraries remain foundational practices rather than optional hardening steps.

Beyond the technical work, the project was managed using Jira for task allocation and progress tracking, GitHub for version controlled code collaboration and Google Drive for shared documentation with quality assurance built into the workflow through peer code review and independent exploit, mitigation and validation between team members. This combination of technical depth and structured project management reflects the dual objective of the assignment which demonstrates both security competency and professional coordination team delivery.

8. References

FLASK-WTF Documentation. (2025). *CSRF protection — Flask-WTF documentation* (1.2.x). Readthedocs.io. <https://flask-wtf.readthedocs.io/en/1.2.x/csrf/>

MITRE. (n.d.). *CWE - CWE-117: Improper output neutralization for logs (4.13)*.

Cwe.Mitre.Org. Retrieved 9 October 2026, from <https://cwe.mitre.org/data/definitions/117.html>

MITRE. (2009). *CWE - CWE-352: Cross-site request forgery (CSRF) (3.4.1)*. Mitre.Org.
<https://cwe.mitre.org/data/definitions/352.html>

MITRE. (2021). *CWE - CWE-93: Improper neutralization of CRLF sequences ('CRLF injection') (4.16)*. Mitre.Org. <https://cwe.mitre.org/data/definitions/93.html>

OWASAP FOUNDATION. (2026). *Cross site request forgery (CSRF) | owasp foundation*. Owasp.Org. <https://community.owasp.org/attacks/csrf>

OWASP. (2012). *Cross-site request forgery prevention · OWASP cheat sheet series*. Owasp.
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP. (2021). *A01:2021 – Broken access control*. OWASP. Owasp.Org.
https://owasp.org/Top10/A01_2021-Broken_Access_Control/

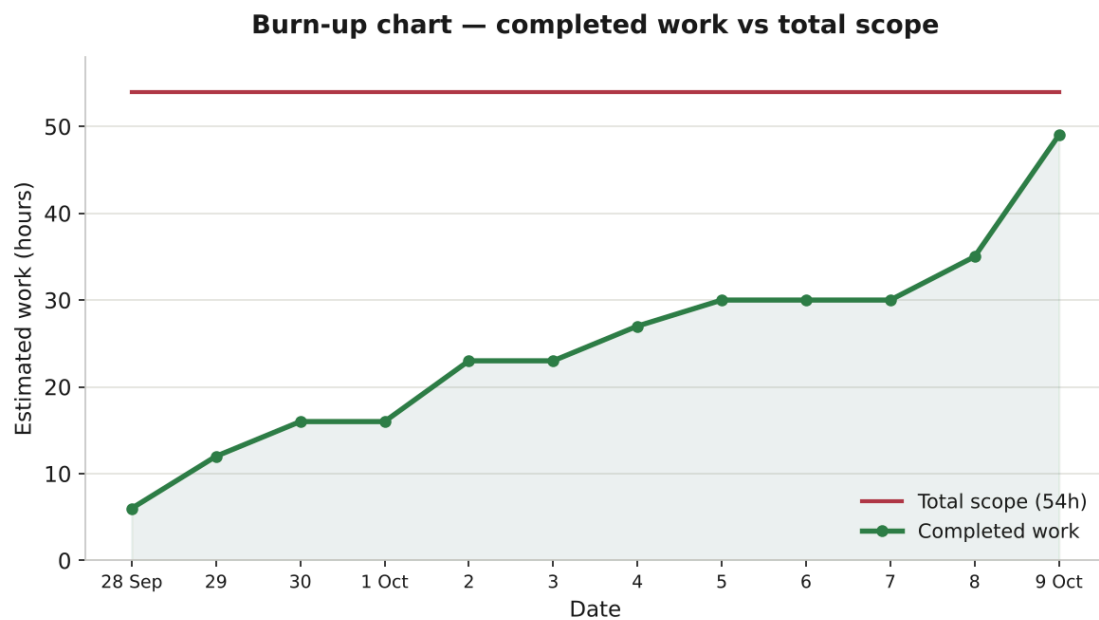
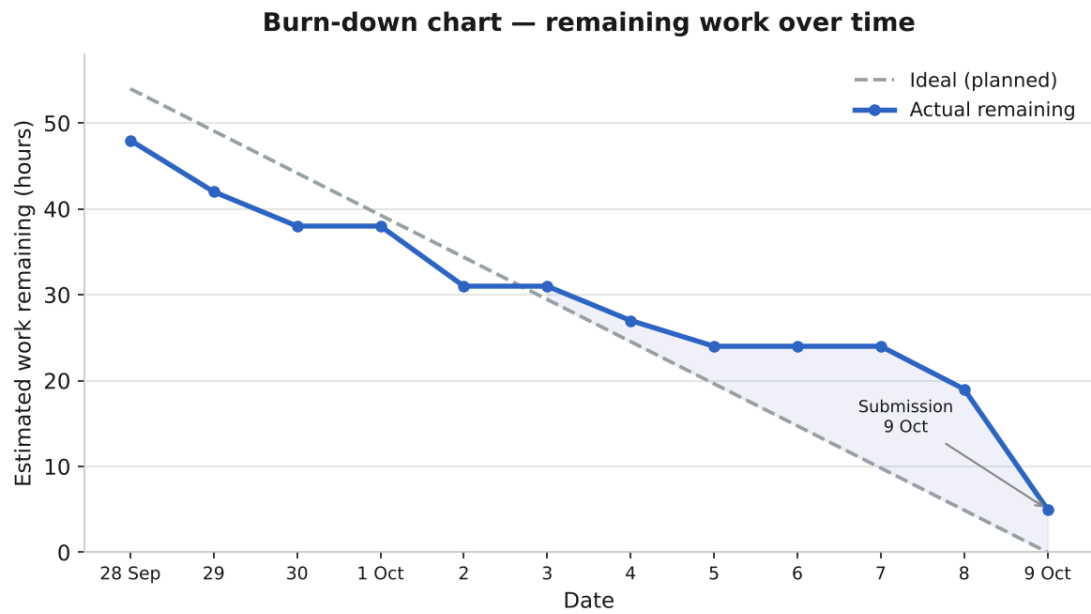
OWASP FOUNDATION. (n.d.-a). *CRLF Injection | OWASP Foundation*. Owasp.Org.
Retrieved 9 October 2026, from
https://owasp.org/www-community/vulnerabilities/CRLF_Injection

OWASP FOUNDATION. (n.d.-b). *Log injection software attack | OWASP foundation*. Owasp.Org. Retrieved 9 October 2026, from
https://owasp.org/www-community/attacks/Log_Injection

9. Appendices

Appendix A Agile Workflow

Task	Owner	Est.	Planned	Actual	Variance
GitHub repo setup	Siluni	1h	16 Sep	16 Sep	On time
Team Org & Project Plan	Sasindu	2h	19 Sep	19 Sep	On time
QA checklist	Sasindu	1h	28 Sep	28 Sep	On time
Research CSRF	Sasindu	2h	29 Sep	28 Sep	-1d (early)
Build CSRF app	Sasindu	4h	30 Sep	29 Sep	-1d (early)
CSRF exploit	Sasindu	4h	30 Sep	30 Sep	On time
Research Log Injection	Siluni	2h	29 Sep	29 Sep	On time
CSRF detection/tracking	Santhosh	3h	2 Oct	2 Oct	On time
Build Log Injection code	Siluni	4h	2 Oct	2 Oct	On time
Log Injection exploit	Siluni	4h	2 Oct	4 Oct	+2d
Intro / Scope	Santhosh	3h	5 Oct	5 Oct	On time
Vuln 1 write-up	Sasindu	2h	4 Oct	8 Oct	+4d
Log Injection detection	Sasindu	3h	4 Oct	8 Oct	+4d
CSRF mitigation	Siluni	5h	5 Oct	9 Oct	+4d
Log Injection mitigation	Santhosh	5h	4 Oct	9 Oct	+5d
Mitigation strategies	Santhosh	2h	5 Oct	9 Oct	+4d
Conclusion	Santhosh	2h	5 Oct	9 Oct	+4d



Appendix B

Jira Workflow Evidence

Jira Board Link - [Jira Board](#)

Screen shots for reference -  PM Evidence

Appendix C

Version Control - Github

<https://github.com/siluni-deshapriya/ISEC3004-Security-Project>

siluni-deshapriya / ISEC3004-Security-Project

Type to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Branches

New branch

Overview

Yours

Active

Stale

All

Q Search branches...

Branch	Updated	Check status	Behind / Ahead	Pull request
main	37 minutes ago		Default	...
mitigation-vul2	2 days ago		8 0	#3 ...
detection	last week		14 0	#1 ...

ISEC3004-Security-Project Public

Watch

main

3 Branches

0 Tags

Go to file

Add file

Code

siluni-deshapriya added readme for log injection 45920c6 · 37 minutes ago 25 Commits

vuln1-CSRF

added readme for log injection37 minutes ago

vuln2-log-injection

added readme for log injection37 minutes ago

ReadMe.md

Add a ReadMe to Main Proj Dir2 hours ago

README

ISEC3004 Assignment 1 - Security Vulnerability Demonstration (Group 6)

About

A set of small apps that demonstrate common security vulnerabilities, each with a working exploit and a mitigated version. Built for ISEC3004 at Curtin University to show how the flaws arise and how to defend against them.

Contributors 3

Sammm2006 SasinduPabasara

siluni-deshapriya

Santhosh19420 Santhosh_Perera

Languages

Python 76.7%

HTML 13.4%

CSS 9.9%

Appendix D

Meeting Notes

 Meeting Notes

Appendix E

Effort Estimation

 Effort Estimation / Roles

Appendix F

Quality Assurance Checklist

 Quality Assurance Checklist & Review Process

Appendix G

Screenshot and evidence standards

 Screenshot and Evidence Standard